SEMTM0051 Foundations of AI
Coursework: underfitting, overfitting and held-out data

Released: Thursday 22 October at 17h. Deadline: Thursday 26 November at 13h.
Weighting: 30% of the unit mark.

What this coursework is about

Neural networks are used to make predictions about unknown data. However, a neural network is trained by making its loss on the training data as small as possible. A natural assumption is that the network with the smallest training loss is the best network. This coursework is about why that is wrong!

In this coursework we will look at the two ways it is wrong to think only about training loss. A network that is too simple to represent the function behind the data has a large training loss and a large loss on new data; this is *underfitting*. A network that is flexible enough to pass close to every training point, including the noise in those points, has a tiny training loss but does badly on new data; this is *overfitting*. In the first case the network is too simple for the data, in the second, the converse. In both cases the quantity we actually care about, how well the network predicts data it has not seen, is not the quantity we trained on, and the only way to measure it is to hold some data back and not train on it. How to do that, and how not to fool yourself while doing it, that is what this exercise is about.

Everything here is done with simulated data, that is, data we generate ourselves from a known function with added noise. This has one enormous advantage over real data: we can generate as much new data as we like, so we can check what the held-out estimates are estimating. It has two disadvantages. The first problem is that it is easy to cheat, because the function and the noise level are sitting in the file that generates the data. This coursework is most useful if you avoid cheating in this way and so there is an informal rule that you treat `data.csv` as if it were measurements from an experiment whose underlying law you do not know, and ideally you should not look at the true function or the noise level until you are told to. Of course, this rule isn't really enforced, it is just recommended.

The second disadvantage with simulated data is one we don't really get into here but which should always be remembered; in designing simulated data we can miss some of the oddness of real data, for example, there is good reason to expect that noise in data follows a normal distribution and when we make simulated data we often use Gaussian noise. However, while there are theorems that tell us that noise is often normally distributed, those theorems don't always hold and sometimes noise has, for example, longer tails than a Gaussian curve. The noise might not be uniform as well, some other attribute of the data might determine the amount of noise, or the noise might be multiplicative rather than additive or vice versa. This is all to say, the real test of any method is its encounter with real data, but simulated data are a great way to explore and test ideas or to develop your intuition.

You should expect the whole thing to take around six to eight hours including the write-up. No single training run takes more than about twenty seconds on a laptop CPU; if something is taking minutes you are doing something wrong!

Setting up

This is what you're used to! You need Python 3 with `numpy`, `matplotlib` and `torch`; all three are available in Google Colab, and `pip install torch` works on most laptops. Download the three files

```
make_data.py     generates the data
fit.py           the network and the training loop
```

```
exercise1.py     a worked example for the first exercise
```

and put them in one directory. Run `python make_data.py` once; it writes `data.csv`, a file with 120 rows of (x, y) pairs. Then run `python exercise1.py`, which should produce a figure with six panels and print six lines, and read `fit.py` from top to bottom. It is short and the exercises will ask you to use every function in it. In the 'basic' version of the coursework you won't have to change this function, but for the bonus questions you will.

The data

Each row of `data.csv` is a pair (x_n, y_n) with x_n in $[-2, 2]$. The y_n were produced as

$$y_n = g(x_n) + \epsilon_n, \qquad \epsilon_n \sim \mathcal{N}(0, \sigma^2),$$

for some smooth function g and some noise level σ, neither of which you know. Our aim is to learn a function $f_{\boldsymbol{\theta}}(x)$ that predicts y from x; the best we could possibly do is $f = g$, and even that would make a mean squared error of σ^2 on new data, because the noise in a new y is not predictable from x. This σ^2 is called the *irreducible error* or the *noise floor*, and one of the things you will do is estimate it without being told it, though, of course, you could cheat and look in `make_data.py`, we are hoping you won't!

The model and the training loop

`fit.make_net(H)` builds a network with one hidden layer of H sigmoid units,

$$f_{\boldsymbol{\theta}}(x) = \sum_{i=1}^{H} w_i^{(2)}\, \sigma\big(w_i^{(1)} x + b_i^{(1)}\big) + b^{(2)},$$

where $\sigma(a) = 1/(1 + e^{-a})$ is the sigmoid function; the network has $3H + 1$ parameters. With $H = 0$ it is just a straight line $f(x) = wx + b$. This is the simplest network that can represent a curve, and the number of hidden units H is the only thing we will change about its architecture; H controls the *capacity* of the model, meaning how complicated a function it is able to represent.

`fit.train(x, y, hidden=H, epochs=E)` trains such a network on the points (x, y) by minimising the mean squared error

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^{N} \big(f_{\boldsymbol{\theta}}(x_n) - y_n\big)^2$$

with the Adam optimiser, using the whole dataset for every gradient step (there is no minibatching because the dataset is tiny). It returns the trained network and a `history` dictionary whose entry `history['train']` is a list of the training loss after every epoch. If you also pass a validation set, `fit.train(..., xval=xv, yval=yv)`, it records `history['val']`, the mean squared error on the validation set after every epoch, which the training never uses for anything except this record. The function `fit.mse(net, x, y)` gives the mean squared error of a trained network on any set of points, and `fit.predict(net, xgrid)` evaluates it on an array of x values so that you can plot the curve.

Two things are worth knowing about this training loop. Neural network training depends on the random initial weights, and `fit.train` takes a `seed` argument so that a given call always produces the same result; the default seed is 0, so when you repeat an experiment with different H you are at least comparing like with like. And the number of epochs matters: the default in the examples, $20\,000$, is deliberately large, large enough that the bigger networks have time to do something they should not, as you will see.

Adam

The gradient descent you have seen so far updates every parameter by the same multiple of its gradient, $\theta_i \leftarrow \theta_i - \eta\,\partial L/\partial\theta_i$. By default `fit.train` uses a variant called *Adam* instead, and since you have not met it, here is what it does. Adam keeps, for each parameter separately, a running average m_i of its recent gradients and a running average s_i of their squares, and takes the step

$$\theta_i \leftarrow \theta_i - \eta\,\frac{m_i}{\sqrt{s_i}+\epsilon},$$

where ϵ is a tiny constant that stops the division blowing up. Dividing by $\sqrt{s_i}$ means the step for each parameter is measured in units of that parameter's own typical gradient, so a parameter whose gradient is consistently tiny, as happens for the input weights of a sigmoid unit that is sitting on one of its flat parts, still takes steps of size about η, and a parameter with huge gradients is not thrown about. The running average m_i smooths the direction, which is the momentum idea. The practical consequences are that Adam usually reaches a given training loss in far fewer steps than plain gradient descent, and that its learning rate is less fiddly to choose: $\eta = 0.01$ or 0.001 works for most problems, whereas the right learning rate for plain gradient descent can be anywhere over several orders of magnitude and the wrong one either does nothing or diverges.

In PyTorch the optimiser is an object that you give the network's parameters to, and it is a drop-in replacement: the only line of `fit.py` that changes between the two is

```
opt = torch.optim.SGD(net.parameters(), lr=lr)     # plain gradient descent
opt = torch.optim.Adam(net.parameters(), lr=lr)    # Adam
```

and in both cases the training loop calls `opt.zero_grad()`, then `loss.backward()` to compute the gradients, then `opt.step()` to update the parameters. PyTorch calls plain gradient descent `SGD`, for stochastic gradient descent, even when, as here, it is given the whole dataset and so is not stochastic at all. `fit.train` takes an argument `optimiser="adam"` or `optimiser="sgd"` that chooses between them, and Exercise 7 asks you to compare them.

Exercises

Exercise 1: fitting all the data

Run `exercise1.py`. It trains networks with $H = 0, 1, 2, 5, 20, 100$ on all 120 points, plots each fitted curve over the data, and prints the number of parameters and the final training loss for each. Include the figure in your report and answer the following.

(a) Describe the fitted curve for $H = 0$ and $H = 1$. In what sense are these networks failing, and what word do we use for it?

(b) Describe the fitted curve for $H = 20$. It passes closer to the training points than the $H = 5$ curve does; explain why you nonetheless would not want to use it to predict y for a new x.

(c) The training loss decreases as H increases. Explain why, in general, it must: that is, why a network with more hidden units can always achieve a training loss at least as small as one with fewer.

(d) Given only the training losses, could you choose the best value of H? Explain why not.

[10 marks]

The resolution to (d) is that the thing we want, the error on data the network has not seen, has to be measured on data the network has not seen. So we divide the data into pieces. The

training set is what we minimise the loss on. The *validation set* is held back from training and used to compare models, for example networks with different H, and to make choices such as when to stop training. The *test set* is held back from everything, used exactly once, at the very end, to report how well the final chosen model does. Why we need two held-out sets rather than one is the subject of Exercise 5; for now, note that `fit.split(x, y, n_train, n_val)` shuffles the data and cuts it into training, validation and test pieces, with the test piece being whatever is left over:

```
(xtr, ytr), (xva, yva), (xte, yte) = fit.split(x, y, n_train=30, n_val=30)
```

gives 30 training points, 30 validation points and 60 test points. Use exactly this split for the rest of the coursework, and from this point on do not call `fit.train` on anything but `(xtr, ytr)`.

Exercise 2: choosing capacity with a validation set

Train networks with $H = 0, 1, 2, 3, 5, 10, 20, 50, 100$ on the training set only, for $20\,000$ epochs, and for each one record the final training loss and the validation loss, `fit.mse(net, xva, yva)`. Plot both against H on one set of axes, with H on a logarithmic scale (plot $H = 0$ at 0.5, or leave it out of the plot and report it in the text).

(a) Describe the shape of the two curves and explain the shape of the validation curve in terms of underfitting and overfitting.

(b) Which H would you choose on the basis of this plot? Be honest about how clear-cut the choice is.

(c) The validation loss cannot, on average, go below the noise floor σ^2. Explain why, and use your plot to give a rough estimate of σ^2.

(d) Repeat the experiment with `seed=1` in `fit.train`, which changes only the initial weights, and comment on how much the validation curve, and your choice of H, change. What does this tell you about how precisely 30 validation points can pin down the best H?

[15 marks]

Capacity is not the only thing that determines whether a network overfits; so does how long you train it. A large network starts from random weights, which represent a nearly flat function, and as training proceeds it fits first the broad shape of the data and only later the fine detail, including the noise. So the same network passes through underfitting, a good fit and overfitting as the epochs go by, and watching the validation loss during training tells you where it is.

Exercise 3: early stopping

Train a network with $H = 20$ for $20\,000$ epochs, passing the validation set so that `history['val']` is recorded, and plot the training and validation loss against epoch on log–log axes.

(a) Describe the two curves. At roughly what epoch is the validation loss smallest, and what are the training and validation losses there and at the end of training?

(b) *Early stopping* means keeping the weights from the epoch at which the validation loss was smallest, rather than the final weights. Explain why this is a way of controlling overfitting, and explain what it has in common with choosing a smaller H.

(c) Plot the fitted curve at the end of training over the training data and, by training again for just the number of epochs you identified in (a), the early-stopped curve. Describe the difference.

[10 marks]

A third way to control overfitting is to change the loss so that the wiggly solutions are penalised. *Weight decay* adds $\frac{\lambda}{2}\|\boldsymbol{\theta}\|^2$ to the loss; a wiggly curve needs large weights, because the slope of $\sigma(wx + b)$ is proportional to w, so penalising the size of the weights pushes the network towards smoother functions. `fit.train` takes a `weight_decay` argument that is λ.

Exercise 4: weight decay

With $H = 20$ and $20\,000$ epochs, train with $\lambda = 0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$ and tabulate the training and validation losses. Plot the fitted curves for $\lambda = 0$, your best λ, and $\lambda = 10^{-1}$.

(a) Explain the behaviour at the two ends of the table in terms of over- and underfitting.

(b) Which λ would you choose? Compare the validation loss you get this way with the best you got by choosing H in Exercise 2 and by early stopping in Exercise 3.

[10 marks]

You have now used the validation set to make three decisions: the value of H, when to stop, and the value of λ. Each time, you looked at the validation loss of several candidates and kept the one with the smallest. That is exactly what we want the validation set for, but it has a consequence: the validation loss of the winner is no longer an honest estimate of how that model does on new data. Among several candidates whose true errors are similar, the one with the smallest validation loss is likely to be one that got lucky on those particular 30 points, so its validation loss is biased downwards. The more candidates you compare, the larger the bias. This is why the test set exists, and why it is used once: it is the only data that played no part in any decision.

Exercise 5: the test set

Pick your final model: a choice of H, of λ and of number of epochs, justified by Exercises 2–4. Train it on the training set and evaluate it, once, on the test set with `fit.mse(net, xte, yte)`.

(a) Report the training, validation and test losses of your final model. Is the test loss larger than the validation loss? Explain, in your own words, why you should expect it to be, on average.

(b) Suppose that, having seen the test loss, you went back and tried a few more values of λ, and reported the test loss of the best. Explain what would be wrong with this.

(c) Why do we not simply use a single held-out set for both choosing the model and reporting its error? Your answer to (a) is most of the answer; be precise about which number would be misleading and in which direction.

[10 marks]

Finally, the luxury of simulated data. The test loss is an estimate of the true error, the mean squared error your model would make averaged over an unlimited supply of new data, and with 60 test points it is a noisy estimate. Here we can check it. The function `make_data.make(n, rng)` generates n fresh points from the same process:

```
import numpy as np, make_data
rng = np.random.default_rng(123)
xbig, ybig = make_data.make(10000, rng)
```

From this point on you are allowed to open `make_data.py` and read the true function and the noise level.

Exercise 6: what the estimates are estimating

(a) Evaluate your final model on the 10 000 fresh points. This is, near enough, its true error. How close were the validation and test losses from Exercise 5 to it, and which was closer?

(b) Compute the mean squared error that the *true* function g makes on the fresh points, that is, `np.mean((make_data.true_function(xbig) - ybig)**2)`. What should this number be, and how does it compare with the estimate of σ^2 you made in Exercise 2(c)?

(c) Evaluate the $H = 20$, $\lambda = 0$, $20\,000$-epoch network from Exercise 3 on the fresh points, and compare with its validation and test losses. Overfitting is sometimes described as the network "memorising the noise"; using your figure from Exercise 3(c), say where in $[-2, 2]$ that network is making its large errors and why.

(d) In a real problem none of this exercise is possible. State, in two or three sentences, what you would and would not be able to conclude about a model from its training, validation and test losses alone.

[15 marks]

Exercise 7: Adam versus plain gradient descent

With $H = 20$, on the training set, train for $20\,000$ epochs with `optimiser="sgd"` and learning rates 0.01, 0.1, 0.3 and 1.0, and with `optimiser="adam"` and learning rates 0.01 and 0.001. Plot all six training-loss curves against epoch on one set of log–log axes.

(a) For each run, report the first epoch at which the training loss drops below 0.2, or that it never does. Comment on how the two optimisers compare and on how sensitive each is to its learning rate.

(b) Describe what happens with plain gradient descent at learning rate 1.0, and explain it.

(c) Plain gradient descent at learning rate 0.01 ends, after $20\,000$ epochs, with a training loss much higher than Adam's and a validation loss lower than Adam's. In the light of Exercise 3, explain why a slow optimiser can look like a good one.

[**10 marks**]

The exercises so far have been done with the one network, the one kind of data and the one notion of test data that `fit.py` and `make_data.py` provide. The last exercise asks you to change one of those and find out whether what you have learned survives the change. This is the part of the coursework where there is no answer we are checking against; it is marked on whether the question you ask is a sensible one, whether the experiment you design actually answers it, and whether you say clearly what you found, including when what you found is that nothing much changed. It is also the part where originality is expected: an investigation that could have been written by everyone in the class, or that simply repeats Exercises 2–4 with one number changed, will get few of the marks however neatly it is done.

Exercise 8: your own investigation

Choose one question of your own about underfitting, overfitting or held-out data, modify `fit.py` or `make_data.py` as needed to answer it, and write it up in at most two pages plus figures: the question, what you changed and why, the experiment, the result and what you conclude from it. Some possible directions with different levels of difficulty are offered as starting points and not as a list to choose from:

- **Depth against width.** Replace the one hidden layer with two or three, keeping the total number of parameters about the same, and ask whether the deeper network over- or underfits differently, and whether early stopping and weight decay act on it in the same way.
- **A different unit.** Replace the sigmoid by a ReLU and repeat the capacity sweep. The fitted curves look quite different; does the validation curve, and does weight decay still have the smoothing effect that the argument in the text gives for the sigmoid?
- **Two inputs.** Make a version where x is two-dimensional and g is a function of both coordinates, so the fitted function is a surface. Does the amount of data needed to avoid overfitting change, and by how much?
- **Different noise.** Change the noise in `make_data.py`: heavier tails than a Gaussian, noise whose size depends on x, or multiplicative noise. Does the noise floor still mean what it meant, and does the squared-error loss remain a sensible choice?
- **Test data from somewhere else.** Generate test points from outside $[-2, 2]$, or from a different distribution over x within it, and ask how well the validation loss predicts the error there. This is the *distribution shift* problem, and the answer is instructive.
- **Real data.** Take a small real regression dataset of your choosing, with one or two inputs, and repeat Exercises 2, 3 and 5 on it. What is different when you no longer know g or σ, and what do you now have to decide that was decided for you before?
- **The split itself.** Vary the sizes of the training and validation sets, or repeat the whole pipeline over several random splits, and quantify how much the chosen H and the reported test loss move. What split would you recommend for 120 points, and why?

Whatever you choose, state the question in one sentence at the top, keep the test set rule from Exercise 5, and if your experiment needs more compute than a few minutes on a laptop, that is a sign to simplify it rather than to run it overnight; we are interested in simple things that are easy to learn from, not complicate things that spin your laptop fan! **[20 marks]**

What to submit

A single PDF of at most six pages including figures for Exercises 1–7, and at most a further three pages including figures for Exercise 8, containing your figures, the tables asked for, and your answers to the lettered questions, numbered as above; and your code, as one `.py` file or one notebook. Thus, to be clear; the code is a separate submission to the pdf answers; the pdf answers should be at most nine page, with no more than six pages for Exercises 1–7. Please aim for clairity rather than content, squeezing in lots of text by having super thin margins, tiny figures and no gaps between questions will not get you higher marks. Axis labels and figure titles should be in the same font size as the rest of the text. The code is not marked for style but it will be run, so it must reproduce your figures from `data.csv`. Marks are for the explanations: a correct figure with no explanation gets little, a clear explanation of an experiment that came out oddly gets most of the marks for that part. Answers should be short; most lettered parts need two to five sentences.

Marking

Marks are as shown against each exercise, for a total of 100, of which 80 are for Exercises 1–7 and 20 for Exercise 8. Within Exercises 1–7, roughly a third of the marks are for the experiment being done correctly and presented clearly, and two thirds for the explanations. The things that lose marks are: training on anything other than the training set after Exercise 1; using the test set more than once or using it to make a choice; describing a figure without explaining it; and explanations that use the words underfitting and overfitting without saying what is being fitted to what. Exercise 8 is marked on the quality of the question, the design of the experiment, the clarity of the conclusion and its originality, in roughly equal parts; a well-posed question with a clean negative result scores as highly as a positive one, and a large experiment with no clear question scores low.